

ETX: Dynamic GPU Megakernels from One Compiler (Brief)

Author: Kailun Chen · Status: v0.4, measured on AMD MI300X · Date: 2026-09-23 · Full design: docs/ETX_Technical_Design.pdf · Repository: github.com/cklxx/etx-megakernel (private)

0. Summary

- ETX is a compiler that fuses a graph of tile-level operators into one persistent GPU kernel. It is DSL-agnostic: tile bodies come from Triton, TileLang, CK, CuTe or hand-written HIP/CUDA. It is chiplet-aware: MI300X’s eight XCDs are a first-class part of the machine model. Dynamic workloads (MoE routing, variable shapes) remain inputs to the schedule, not reasons to fall back to separate kernels.
- It adopts the Event Tensor abstraction of the ETC paper (arXiv 2604.13327) and adds two things the paper lacks: an explicit machine model, with hardware differences expressed only as data, and a cost model that chooses static, dynamic or hybrid scheduling per subgraph and prints its reasons.
- The proof is deliberately narrow: one real model on one machine against a known hand-written result. The target is DeepSeek-Coder-V2-Lite at batch 1 on one MI300X, where the hand-written fleet-mi300x megakernel runs at 3.556 ms/token.
- **Result:** ETX, driving fleet’s unchanged tile bodies through a shim, decodes 32/32 tokens identical to HuggingFace greedy with all 27 layers inside the accuracy gate, at **3.70 ms/token**. That is within 4% of the hand-written kernel, down from 5.42 ms at first light. Every tile body now runs as fast as fleet’s or faster. The remaining 4% is in three phase handoffs per layer.
- Baselines requested by review are measured: launch, empty-loop and unfused costs; per-step fixed cost reduced from 101 us to 12-23 us; cross-device sync of about 1 us one-way when idle, 2-3 us under load.

1. Problem

Small-batch decoding issues hundreds of short kernels per token. The shortest take about 2 us, while launch gaps are 5-10 us. CUDA/HIP graphs remove the launch gap but keep every kernel boundary as an implicit global barrier. A megakernel replaces those barriers with tile-level dependencies, so a tile of operator B starts as soon as the tiles of A it needs are done.

Hand-written megakernels prove the gain but are not portable: fleet-mi300x encodes MI300X’s XCD topology, cache rules and task placement by hand. The ETC paper shows a compiler can generate such kernels. It also shows the risk: the same dynamic scheduler that wins 4% on one MoE layer loses 17% on a dense tensor-parallel model. The scheduling policy must therefore be a cost-model decision. The paper also assumes one coherent L2, which is false on MI300X.

2. Architecture

2.1 Five layers

Layer	Role	Key content
L1 Tile contract	What a tile body must declare	Link mode (a C ABI: coordinate, shape, argument table, events, domain, LDS, per-grid constants) or host-DSL mode

Layer	Role	Key content
L2 Event Tensor IR	Dependencies as counter tensors	Edge maps such as "ij->i", "i->range(indptr[i],indptr[i+1])", "i->topk[i,:]" ; scope, domain, init kind, epoch; six verification checks
L3 Machine model	Hardware as data	Exec-domain tree (device > XCD > CU), visibility table (fence / poll / arrive per scope), capability table (legality), cost table (measured latencies)
L4 Passes	Placement and schedule	P1 tiling and worker count, P2 domain affinity, P3 event elimination by recomputation, P4 schedule mode, P5 queues and relay, P6 memory, P7 cross-barrier prefetch
L5 Lowering and runtime	Generated persistent kernel	Worker loop, per-scope primitives from the lowering table, ticket queues, host launcher with watchdog and per-phase trace

2.2 Three hard decisions

1. **Hardware differences exist only as data.** Every architecture is a YAML file (gfx942, gfx950, gfx90a, sm_90, sm_100). A test fails if a pass or code generator names an architecture.
2. **Capability and cost are separate.** Capabilities decide what is legal. Costs decide what is worthwhile. A wrong cost gives a slow kernel; a wrong capability would give a wrong one, so the two never mix.
3. **The schedule is chosen by a cost model that explains itself.** Every grid gets static, dynamic or hybrid scheduling with a printed reason in `decisions.log`. Hybrid means dynamic queues inside one XCD and pre-ordered static queues with event counters across XCDs, so no push crosses a domain.

2.3 Performance model

A fused kernel has two lower bounds: bytes moved divided by bandwidth, and the critical path, which is the sum over dependent phases of body time plus fixed synchronisation latency. At batch 1 the second dominates. In fleet's MoE layer about 31 us of a 126 us layer is data movement and the rest is per-phase fixed latency. ETX therefore treats four levers as compiler passes: fewer phases (event elimination by recomputation), cheaper signals (domain-local events, last-arriver flush), cross-barrier weight prefetch, and fine-grained events.

3. Measured Results (MI300X, ROCm 7.2.4)

3.1 Baselines and synchronisation costs

Measurement	Value
Cooperative launch, empty 608-workgroup kernel	16 us
Ordinary launch	1.6 us
Empty persistent worker loop	28 us
Unfused two-kernel split-K step	5.7 us
ETX split-K step, v0.3 runtime	101 us
ETX split-K step, v0.4 runtime (cooperative / ordinary launch)	23 / 12 us
One-way counter signal, same XCD / cross XCD	642 / 742 ns
Whole-GPU counter barrier, 304 workgroups	8.6 us
Cross-device signal, flag in consumer memory, idle / loaded	1 us / 2-3 us

The v0.3 fixed cost came from a per-task completion atomic and atomic idle polling, about 15 us per task slot. v0.4 counts only dynamic tasks, exits when the static queue drains, and polls with plain scoped loads.

3.2 The real model: progression to 3.70 ms/token

Same model, same VM class, fleet's tile code unchanged. Each row adds one compiler or runtime rule.

Step	Rule added	ms/token
First light	Fleet's task graph in ETX, fleet's tiles through a shim	5.42
Worker pinning, last-arriver flush	Keep each expert's gate_up and down on one worker; one L2 write-back per XCD per global event	4.81
Agent-scope polling	The poll that was fastest idle lost 5% under load	4.60
In-body q_c wait	Attention waits for q_c inside its body, as fleet does	4.49
One acquire per workgroup	The L2 invalidate issued once, not by all four waves	4.09
One call site per tile body; lean static loop; layer as immediate	Emitter restructured into wait switch, single body call, arrive switch	3.90
Whole-program device build	No -fgpu-rdc; scratch 112 to 0 B/lane	3.70
Fleet, hand-written, same VM		3.556

Levers measured and found neutral or worse on this model:

- **Per-XCD relay of global counters:** neutral (3.90 vs 3.88). It is kept because it only breaks even through its hierarchical acquire; with a per-consumer L2 invalidate under the relay the time was 4.04.
- **Fleet parameters in constant memory:** no measurable effect.
- **Poll backoff:** s_sleep values 1, 3 and 8 gave 3.74, 3.73 and 3.71; 8 is the default.
- **38 workers per XCD without a relay:** 3.78, slower than 37 plus relay.
- **Descriptors staged in LDS:** 3.72, reverted.

3.3 Where the last 4% is

Traced layer span is 130.8 us against fleet's 126.9 us. Per-task body times now match or beat fleet's (router 8.5 vs 10.7 us, gate_up 31.3 vs 33.6, down 16.1 vs 18.0, qkv 18.6 vs 17.1). The gap is in three handoffs per layer, after o_proj, the router and down, each 2-3 us where fleet shows about 1 us. Fleet also decodes 32 tokens in one launch with epoch counters; ETX launches once per token.

3.4 Other examples on hardware

Example	Result
Split-K row sum, static / hybrid / dynamic	Matches the CPU reference in all three modes
Full MoE layer, device-side top-2 routing	Routing identical, outputs within 3.5e-5
Short-task MoE, B=64	Static 0.445 ms vs hybrid 0.654 ms; the cost model now selects static
GEMM + reduce-scatter across two MI300X	Correct, 0.365 ms per step, system-scope events over xGMI

4. Rules Learned Only on Hardware

These defects and costs were invisible in simulation. Each is now a pass rule, runtime rule or test.

- **Queues:** ticket-ring slots carry a pass tag, otherwise wrap-around re-executes consumed tasks. Multi-input consumers use remaining-dependency counters, otherwise they are pushed twice.
- **Scope:** grids scheduled across XCDs need device-scope events. A domain-scope event under global scheduling returned stale data.
- **Signals:** one L2 write-back per XCD per global event (last-arriver flush); one acquire per workgroup, not per wave; poll with agent-scope atomics, because a poll that is faster idle can be slower under load.
- **Code shape:** each tile body must have one call site. A non-inlined helper taking the parameter block by reference spills the whole block to scratch. Each of these cost about 0.2 ms/token, more than any synchronisation choice.
- **Placement:** discover the workgroup-to-XCD mapping from the hardware ID register. It was $(k+6) \bmod 8$, not the documented round-robin.

5. Status and Next Steps

Milestone	Status
M0 Baselines	Done
M1 Fleet graph in ETX	Done
M2 Fleet tiles through the shim, HF-exact	Done
M3 Match fleet within 3%	4% (3.70 vs 3.556 ms/token)
M4 Generalise	Not started

Next, in order:

1. **Close M3:** multi-token launches with epoch counters instead of per-step resets, then the three remaining handoffs.
2. **M4, generalise:** - a second model, dense or a larger MoE, with ETX-scheduled but not hand-placed tasks; - per-worker queues for the dynamic path, which costs about 50 us per task slot today; - a second architecture from YAML only (gfx950 or an NVIDIA machine; the CUDA backend is untested).

Main risks:

- **Tile bodies are borrowed, not generated.** The 3.70 result reuses fleet's tuned tiles. A fusion-gain figure for arbitrary models needs tuned tiles from a DSL frontend.
- **The dynamic path is expensive.** On short tasks its queue overhead exceeds the gain, so the cost model must keep choosing static where it applies.
- **Portability claims are untested beyond MI300X.**